

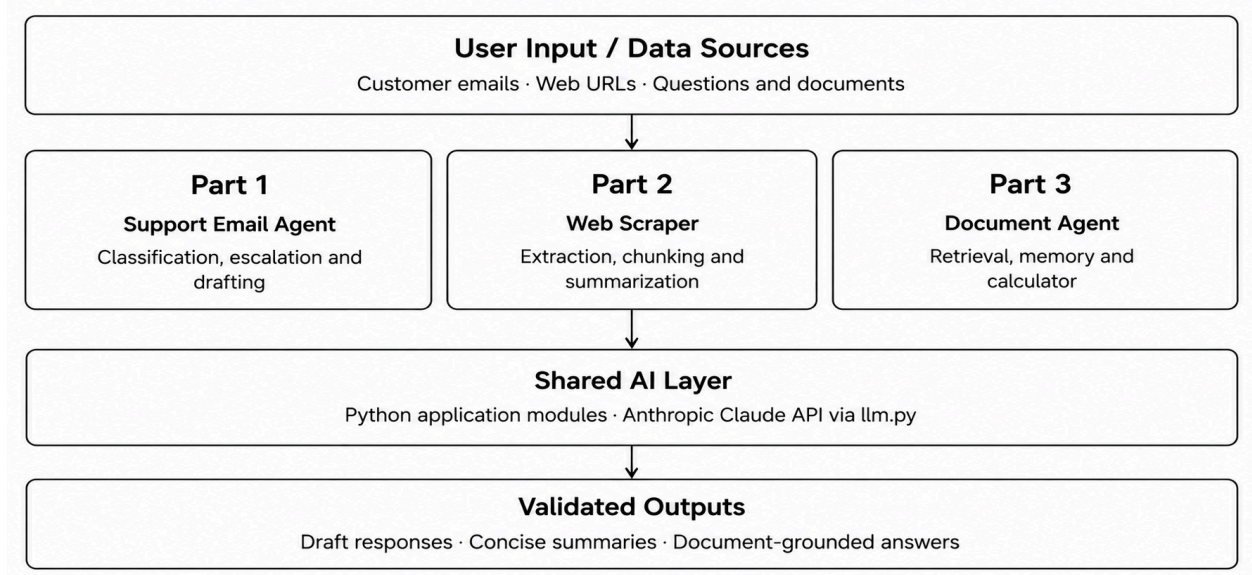
Agentic Architect Challenge

System Architecture, Design Decisions & Failure Handling

1. System Overview

The project has three separate AI-driven workflows written in Python with the Anthropic Claude API. The three workflows solve unique problems: customer support automation, extraction and summarisation of documents on the web, and Q&A in the document.

2. System Architecture



This diagram shows the logical components. The three workflows operate independently and share the LLM integration module.

3. Component Design

Part 1. Customer Support Automation

- Classifies incoming emails into Billing, Technical, Feedback, and other categories.
- Ensures that the following critical conditions are checked beforehand: Data loss, service outage, security breach, or more than three contacts in seven days.
- Raises critical cases for human review.
- Applies the knowledge base to write answers and doesn't make assumptions about refund policies.

Part 2. Website Scarper and Summarizer

- Validates URLs and downloads web pages with Requests for http/https.
- Scrapes readable text and common page noise with BeautifulSoup.
- Breaks down longer text into segments and summarises them with Claude.
- Implements a maximum summary length of 120 words, default is 120 words.

Part 3. Document-Based AI Agent

- Locates paragraphs in a text based on keywords.
- Saves information related to the conversation history and the name of the user from his session.
- If an arithmetic request is detected, it uses an arithmetic calculator with limited functionality in Python.
- Tells Claude what to answer and what information is missing.

4. Technology Stack

Component	Technology
Programming Language	Python
LLM	Anthropic Claude API
HTTP requests	Requests
HTML parsing	Beautiful Soup
Arithmetic tool	Python AST and operator
Testing	Pytest
Configuration	Environment variables

5. Design Trade-offs and Failure Handling

Design Choice	Benefit	Limitation and Mitigation
Rule-based Classification	Fast and Predictable	May miss keyword variations; expand rules and retain human review.
Keyword Retrieval	Simple and inexpensive	MAY miss semantic matches; consider embeddings for larger documents.
Chunk-based Summarization	Support long pages	May lose context across chunks; enforce a final word limit and test summaries.
Requests and Beautiful Soup	Lightweight and easy to run	Cannot render JavaScript; add browse automation if required
Restricted Calculator	Avoids arbitrary code execution	Supports limited arithmetic; reject unsupported expressions

6. Testing and Operational Considerations

- Automated testing: Passed 20 tests in all three modules: 3 with support; 9 with scraper; 8 with document-agent.
- Manual verification: Tested critical support escalation, classification, knowledge base drafting, webpage extraction and summarizing long content.
- Security: API keys are stored as environment variables and should never be added to git hub.
- Future enhancements: Structured logging, retries, monitoring, expanded test coverage, and advanced document retrieval.